

Start coding or [generate](#) with AI.

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call

First, let's list the contents of your Google Drive to help you find the path to your CSV file.

```
import os

# List contents of 'My Drive'
!ls '/content/drive/My Drive/'

'239X1A3397-Application (1).pdf'
'239X1A3397-Application (2).pdf'
239X1A3397-Application.pdf
239x1a3397.pdf
book.gdoc
booking.txt
cinema.css
Cinema.html
'Colab Notebooks'
DOC-20251017-WA0027.pdf
frame31.gdoc
frame31.pdf
frame3.txt
frame4.gdoc
google.pdf
IMG-20240913-WA0006.jpg
IMG-20241005-WA0003.jpg
IMG-20241105-WA0023.jpg
'KLS Resume(2).pdf'
Resume.gdoc
Screenshot_20241107_181747_PhonePe.jpg
'Screenshot 2025-10-17 at 10-31-40 JavaScript Events – Demo.png'
semantics.gdoc
'Student certificate.gslides'
style.gdoc
'Tanagala_Shireesha_ATS_Resume (25).docx'
'Tanagala_Shireesha_ATS_Resume_Fixed (1).pdf'
'TANAGALA SHIREESHA resume.docx'
'TEAM NUMBERS.pptx'
'THE FESTIVAL OF TIME'S EMBRACE (1).pptx'
'THE FESTIVAL OF TIME'S EMBRACE.pptx'
'total (1).txt'
total.txt
'Untitled document (1).gdoc'
'Untitled document (2).gdoc'
'Untitled document.gdoc'
'Untitled presentation.gslides'
'Untitled spreadsheet.gsheets'
```

'Untitled spreadsheet.xlsx'

'White and Green Simple Student CV Resume_20241022_151839_0000.png'

```
import pandas as pd
```

```
# Replace 'your_csv_file_path.csv' with the actual path to your CSV file in
csv_file_path = '/content/drive/My Drive/your_csv_file_path.csv'
df = pd.read_csv(csv_file_path)
```

```
# Display the first 5 rows of the DataFrame
display(df.head())
```

```
-----
FileNotFoundError                                Traceback (most recent call last)
/tmp/ipykernel_2421/1942992989.py in <cell line: 0>()
      3 # Replace 'your_csv_file_path.csv' with the actual path to your CSV
file in Google Drive
      4 csv_file_path = '/content/drive/My Drive/your_csv_file_path.csv'
----> 5 df = pd.read_csv(csv_file_path)
      6
      7 # Display the first 5 rows of the DataFrame
```

4 frames

```
/usr/local/lib/python3.13/dist-packages/pandas/io/common.py in
get_handle(path_or_buf, mode, encoding, compression, memory_map, is_text,
errors, storage_options)
    871         if ioargs.encoding and "b" not in ioargs.mode:
    872             # Encoding
--> 873             handle = open(
    874                 handle,
    875                 ioargs.mode,
```

```
FileNotFoundError: [Errno 2] No such file or directory: '/content/drive/My
Drive/your_csv_file_path.csv'
```

Once you know the path to your CSV file (e.g., [/content/drive/My Drive/my_data.csv](#)), replace `your_csv_file_path.csv` in the code below with the actual path and run the cell to load it into a pandas DataFrame.

```
import pandas as pd
```

```
# Read the uploaded Parquet file
# Corrected path based on available files in the Colab environment
df = pd.read_parquet('/content/UNSW_NB15_training-set.parquet')
```

```
# Display first 5 rows
df.head()
```

	dur	proto	service	state	spkts	dpkts	sbytes	dbytes	rate	
0	0.121478	tcp	-	FIN	6	4	258	172	74.087486	14
1	0.649902	tcp	-	FIN	14	38	734	42014	78.473373	8
2	1.623129	tcp	-	FIN	8	16	364	13186	14.170161	1
3	1.681642	tcp	ftp	FIN	12	12	628	770	13.677108	2
4	0.449454	tcp	-	FIN	10	6	534	268	33.373825	8

5 rows × 36 columns

```
import pandas as pd
```

```
# Load the UNSW-NB15 training dataset into 'df'
```

```
df = pd.read_parquet('/content/UNSW_NB15_training-set.parquet')
```

```
# Display the first few rows to confirm loading
```

```
display(df.head())
```

```
-----
FileNotFoundError                                Traceback (most recent call last)
/tmp/ipykernel_422/1206990341.py in <cell line: 0>()
      2
      3 # Load the UNSW-NB15 training dataset into 'df'
----> 4 df = pd.read_parquet('/content/UNSW_NB15_training-set.parquet')
      5
      6 # Display the first few rows to confirm loading
```

3 frames

```
/usr/local/lib/python3.13/dist-packages/pandas/io/common.py in
get_handle(path_or_buf, mode, encoding, compression, memory_map, is_text,
errors, storage_options)
    880         else:
    881             # Binary mode
--> 882             handle = open(handle, ioargs.mode)
    883             handles.append(handle)
    884
```

```
FileNotFoundError: [Errno 2] No such file or directory:
'/content/UNSW_NB15_training-set.parquet'
```

```
# Check dataset shape
```

```
print("Dataset Shape:", df.shape)
```

```
# Check column names
```

```
print("\nColumn Names:")
```

```
print(df.columns.tolist())
```

```
# Check data types and non-null values
```

```
print("\nDataset Information:")
```

```
df.info()
```

Dataset Shape: (175341, 36)

Column Names:

```
['dur', 'proto', 'service', 'state', 'spkts', 'dpkts', 'sbytes', 'dbytes', 'r
```

Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 175341 entries, 0 to 175340

Data columns (total 36 columns):

#	Column	Non-Null Count	Dtype
0	dur	175341 non-null	float32
1	proto	175341 non-null	category
2	service	175341 non-null	category
3	state	175341 non-null	category
4	spkts	175341 non-null	int16
5	dpkts	175341 non-null	int16
6	sbytes	175341 non-null	int32
7	dbytes	175341 non-null	int32
8	rate	175341 non-null	float32
9	sload	175341 non-null	float32
10	dload	175341 non-null	float32
11	sloss	175341 non-null	int16
12	dloss	175341 non-null	int16
13	sinpkt	175341 non-null	float32
14	dinpkt	175341 non-null	float32
15	sjit	175341 non-null	float32
16	djit	175341 non-null	float32
17	swin	175341 non-null	int16
18	stcpb	175341 non-null	int64
19	dtcpb	175341 non-null	int64
20	dwin	175341 non-null	int16
21	tcprtt	175341 non-null	float32
22	synack	175341 non-null	float32
23	ackdat	175341 non-null	float32
24	smean	175341 non-null	int16
25	dmean	175341 non-null	int16
26	trans_depth	175341 non-null	int16
27	response_body_len	175341 non-null	int32
28	ct_src_dport_ltm	175341 non-null	int8
29	ct_dst_sport_ltm	175341 non-null	int8
30	is_ftp_login	175341 non-null	int8
31	ct_ftp_cmd	175341 non-null	int8
32	ct_flw_http_mthd	175341 non-null	int8
33	is_sm_ips_ports	175341 non-null	int8
34	attack_cat	175341 non-null	category
35	label	175341 non-null	int8

dtypes: category(4), float32(11), int16(9), int32(3), int64(2), int8(7)

memory usage: 17.1 MB

```
# Check missing values
print("Missing Values:")
print(df.isnull().sum())
```

```
# Check duplicate rows
```

```
print("\nNumber of Duplicate Rows:")
print(df.duplicated().sum())

# Basic statistical summary
print("\nStatistical Summary:")
display(df.describe())
```

Missing Values:

```

dur          0
proto        0
service      0
state        0
spkts        0
dpkts        0
sbytes       0
dbytes       0
rate         0
sload        0
dload        0
sloss        0
dloss        0
sinpkt       0
dinpkt       0
sjit         0
djit         0
swin         0
stcpb        0
dtcpb        0
dwin         0
tcprrt       0
synack       0
ackdat       0
smean        0
dmean        0
trans_depth  0
response_body_len 0
ct_src_dport_ltm 0
ct_dst_sport_ltm 0
is_ftp_login 0
ct_ftp_cmd   0
ct_flw_http_mthd 0
is_sm_ips_ports 0
attack_cat   0
label        0
dtype: int64

```

Number of Duplicate Rows:
78519

Statistical Summary:

	dur	spkts	dpkts	sbytes	dbytes
count	175341.000000	175341.000000	175341.000000	1.753410e+05	1.753410e+05
mean	1.359389	20.298664	18.969591	8.844844e+03	1.492892e+04
std	6.483313	136.887597	110.258271	1.747656e+05	1.436542e+05
min	0.000000	1.000000	0.000000	2.800000e+01	0.000000e+00
25%	0.000008	2.000000	0.000000	1.140000e+02	0.000000e+00
50%	0.001582	2.000000	2.000000	4.300000e+02	1.640000e+02
75%	0.668069	12.000000	10.000000	1.418000e+03	1.102000e+03

```
# Count Normal and Attack records
print("Label Distribution:")
print(df['label'].value_counts())

# Percentage distribution
print("\nPercentage Distribution:")
print(df['label'].value_counts(normalize=True) * 100)
```

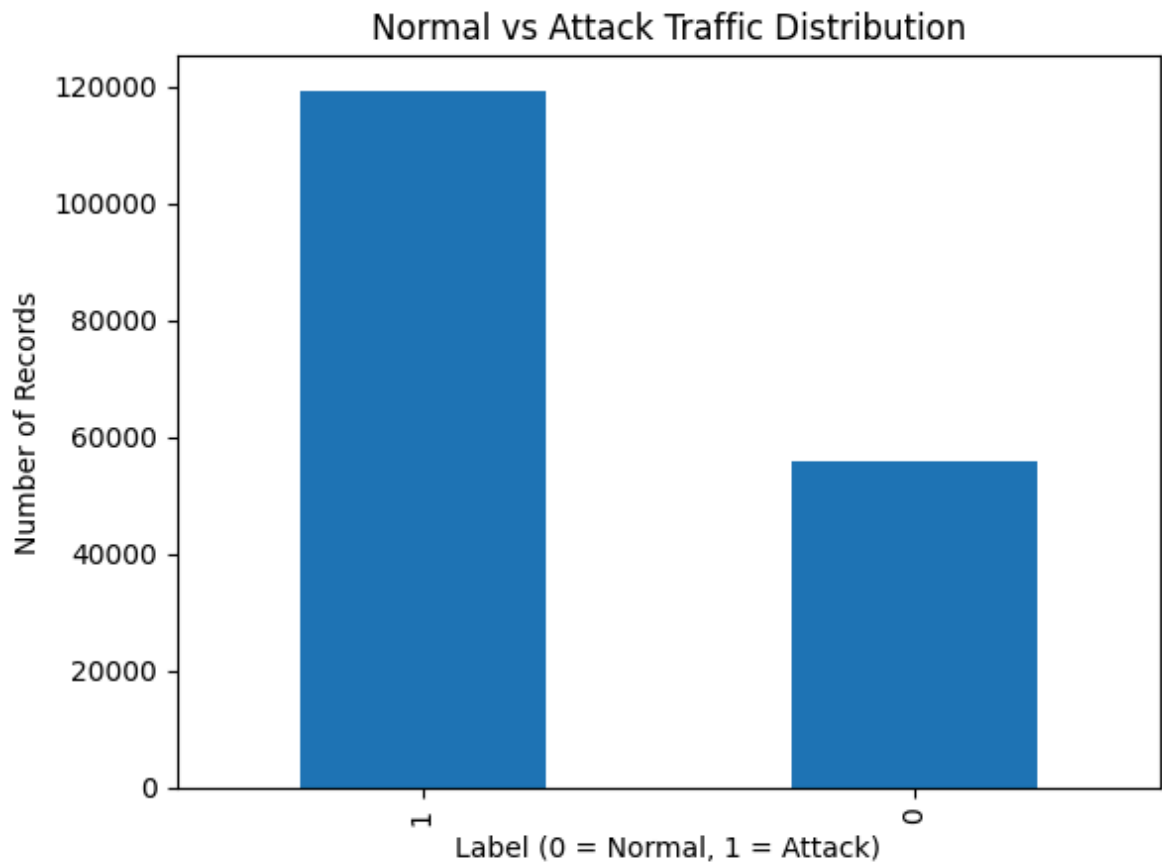
```
Label Distribution:
label
1      119341
0       56000
Name: count, dtype: int64
```

```
Percentage Distribution:
label
1      68.062233
0      31.937767
Name: proportion, dtype: float64
```

```
import matplotlib.pyplot as plt

# Plot Normal vs Attack distribution
df['label'].value_counts().plot(
    kind='bar',
    title='Normal vs Attack Traffic Distribution'
)

plt.xlabel('Label (0 = Normal, 1 = Attack)')
plt.ylabel('Number of Records')
plt.show()
```



```
# Check different attack categories
print("Attack Category Distribution:")
print(df['attack_cat'].value_counts())
```

Attack Category Distribution:

```
attack_cat
Normal          56000
Generic         40000
Exploits        33393
Fuzzers         18184
DoS             12264
Reconnaissance  10491
Analysis        2000
Backdoor        1746
Shellcode       1133
Worms           130
Name: count, dtype: int64
```

```
import matplotlib.pyplot as plt

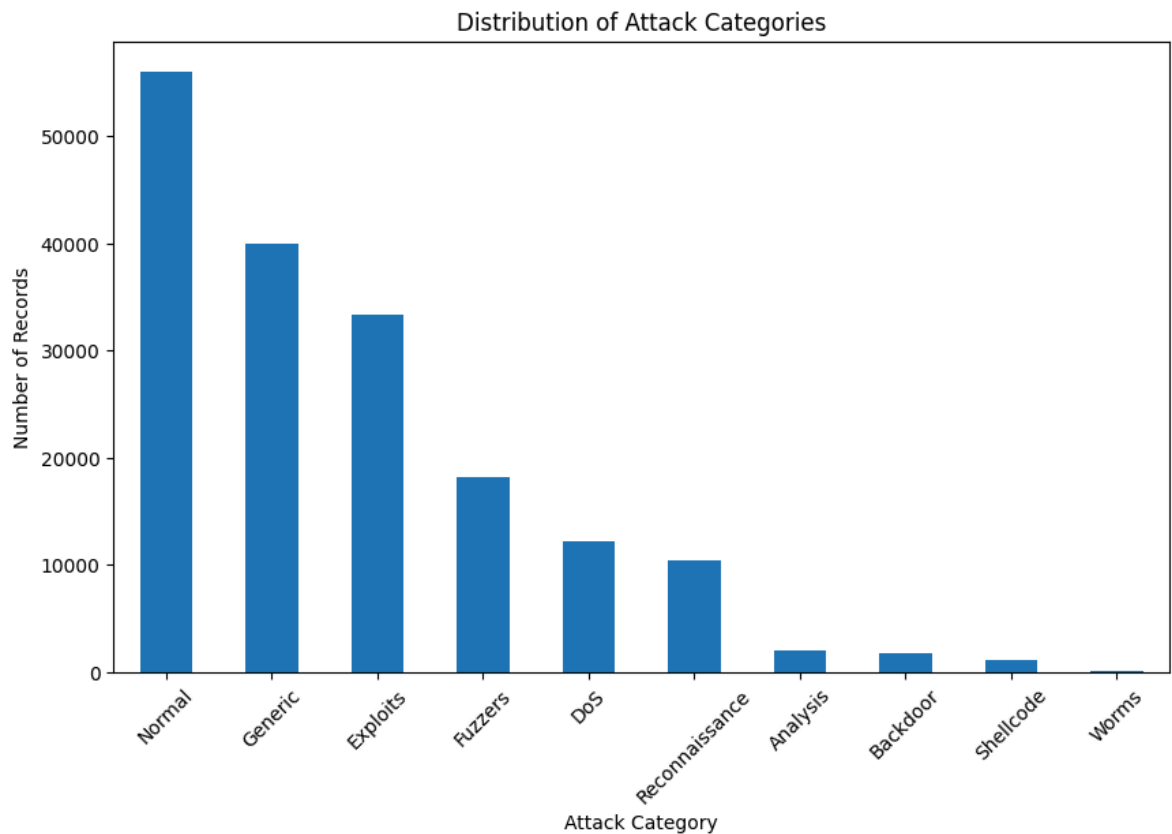
# Count attack categories
attack_counts = df['attack_cat'].value_counts()

plt.figure(figsize=(10, 6))
attack_counts.plot(kind='bar')

plt.title('Distribution of Attack Categories')
plt.xlabel('Attack Category')
plt.ylabel('Number of Records')
```



```
plt.xticks(rotation=45)
plt.show()
```



```
# Protocol distribution
print("Protocol Distribution:")
print(df['proto'].value_counts().head(10))
```

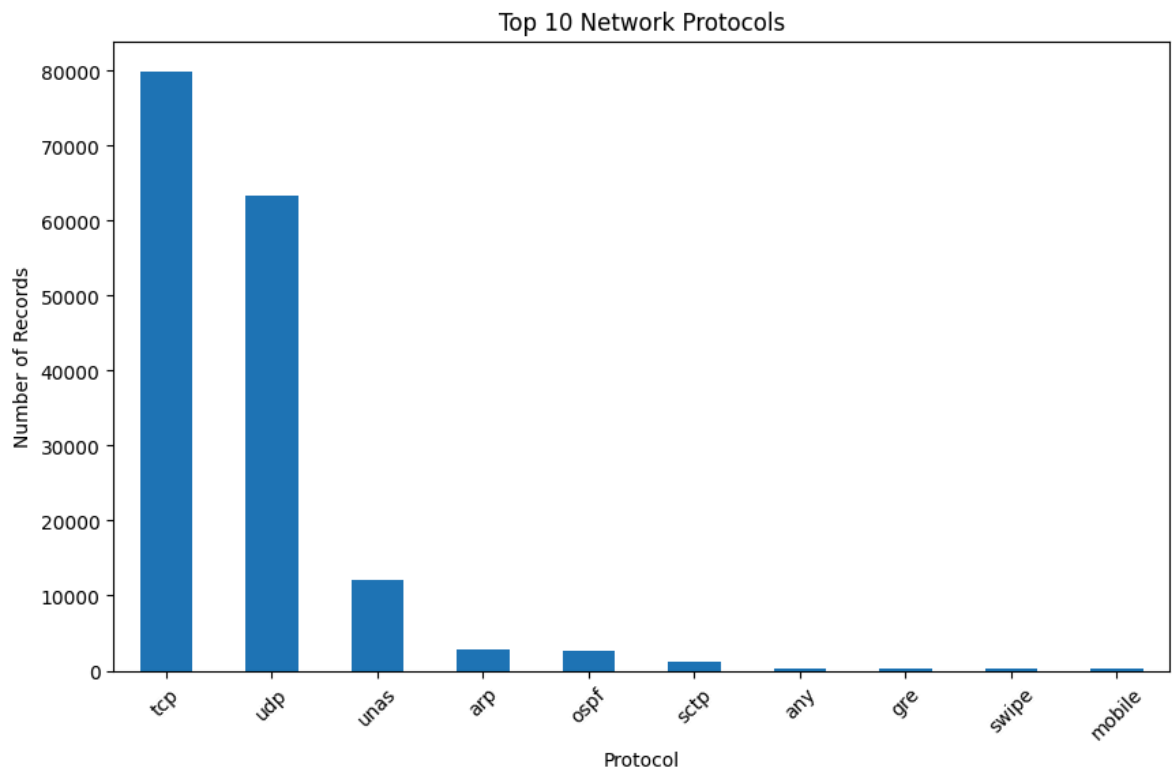
```
Protocol Distribution:
proto
tcp      79946
udp      63283
unas     12084
arp       2859
ospf      2595
sctp     1150
any        300
gre        225
swipe      201
mobile     201
Name: count, dtype: int64
```

```
# Top 10 network protocols
proto_counts = df['proto'].value_counts().head(10)

plt.figure(figsize=(10, 6))
proto_counts.plot(kind='bar')

plt.title('Top 10 Network Protocols')
plt.xlabel('Protocol')
```

```
plt.ylabel('Number of Records')  
plt.xticks(rotation=45)  
plt.show()
```



```
# Top 10 network services  
print("Service Distribution:")  
print(df['service'].value_counts().head(10))
```

```
Service Distribution:  
service  
-          94168  
dns        47294  
http       18724  
smtp       5058  
ftp-data   3995  
ftp        3428  
ssh        1302  
pop3       1105  
dhcp        94  
snmp        80  
Name: count, dtype: int64
```

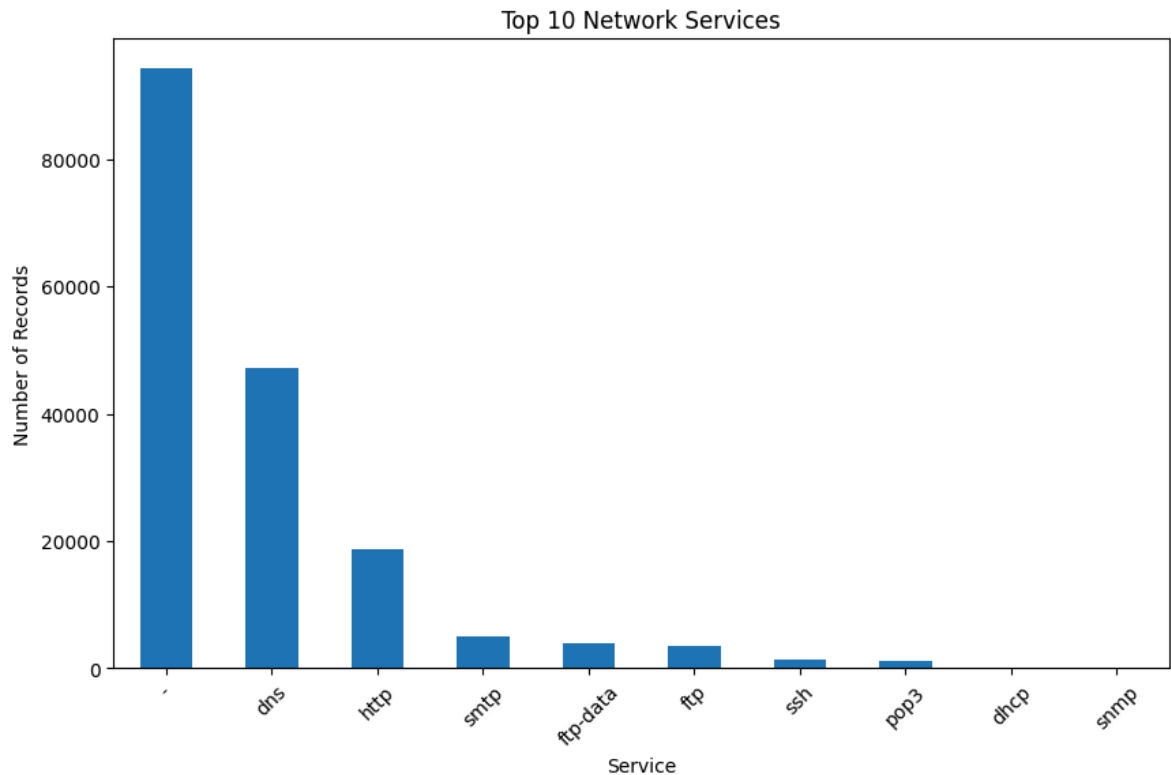
```

service_counts = df['service'].value_counts().head(10)

plt.figure(figsize=(10, 6))
service_counts.plot(kind='bar')

plt.title('Top 10 Network Services')
plt.xlabel('Service')
plt.ylabel('Number of Records')
plt.xticks(rotation=45)
plt.show()

```



```

# Network state distribution
print("Network State Distribution:")
print(df['state'].value_counts())

```

```

Network State Distribution:
state
INT      82275
FIN      77825
CON      13152
REQ       1991
RST         83
ECO         12
PAR          1
URN          1
no           1
Name: count, dtype: int64

```

```

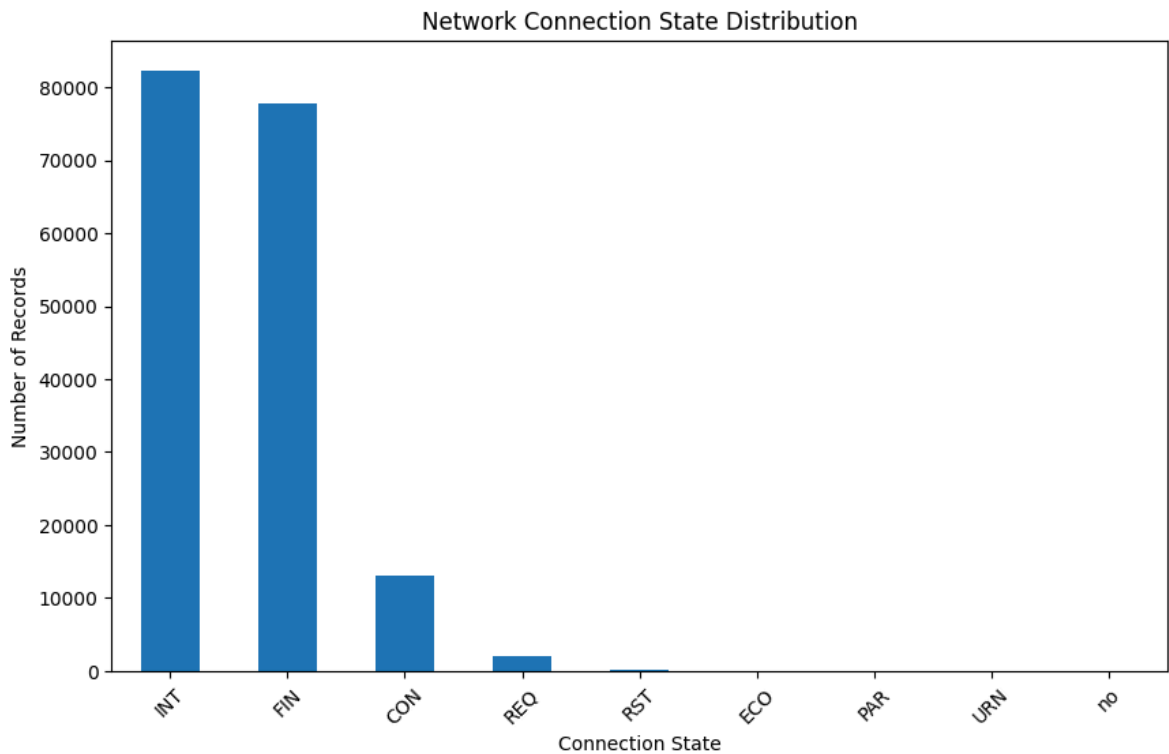
state_counts = df['state'].value_counts()

plt.figure(figsize=(10, 6))

```

```
state_counts.plot(kind='bar')

plt.title('Network Connection State Distribution')
plt.xlabel('Connection State')
plt.ylabel('Number of Records')
plt.xticks(rotation=45)
plt.show()
```



```
import matplotlib.pyplot as plt

# Select only numerical columns
numeric_df = df.select_dtypes(include=['number'])

# Create correlation matrix
correlation = numeric_df.corr()

# Display correlation matrix
correlation
```

	dur	spkts	dpkts	sbytes	dbytes	rat
dur	1.000000	0.254559	0.181182	0.199731	0.144134	-0.12096
spkts	0.254559	1.000000	0.390067	0.963791	0.206609	-0.07635
dpkts	0.181182	0.390067	1.000000	0.188476	0.971907	-0.09820
sbytes	0.199731	0.963791	0.188476	1.000000	0.009926	-0.02846
dbytes	0.144134	0.206609	0.971907	0.009926	1.000000	-0.05947
rate	-0.120966	-0.076358	-0.098202	-0.028468	-0.059475	1.00000
sload	-0.081749	-0.051646	-0.066710	-0.018322	-0.040430	0.60249
dload	-0.050603	0.075897	0.139145	-0.007829	0.104757	-0.15305
sloss	0.198597	0.971069	0.204883	0.996109	0.017366	-0.04292
dloss	0.142963	0.207798	0.978636	0.006804	0.996504	-0.07525
sinpkt	0.080055	-0.017587	-0.022160	-0.006565	-0.013618	-0.07574
dinpkt	0.152142	-0.001678	-0.006514	0.000024	-0.007701	-0.05153
sjit	0.144413	-0.000384	0.000229	-0.002054	-0.002422	-0.06337
djit	0.157443	0.017096	0.054371	-0.003516	0.047354	-0.08580
swin	0.022047	0.131813	0.183703	0.050450	0.113148	-0.51568
stcpb	0.013183	0.107410	0.144119	0.043164	0.086894	-0.40875
dtcpb	0.014724	0.102161	0.142667	0.037988	0.086453	-0.40904
dwin	0.017527	0.133102	0.185555	0.050981	0.114269	-0.51811
tcprtt	0.053125	0.039187	0.020915	0.043624	0.003907	-0.30079
synack	0.051093	0.035507	0.015936	0.039739	0.000101	-0.27927
ackdat	0.049332	0.038725	0.023899	0.042883	0.007546	-0.29005
smean	0.090028	0.216592	0.014697	0.232348	-0.036635	-0.11323
dmean	0.025336	0.150237	0.441445	-0.004973	0.419965	-0.27332
trans_depth	0.002071	0.008834	0.029042	0.003428	0.030912	-0.07855
response_body_len	0.078915	0.087217	0.442194	0.001620	0.470905	-0.02275
ct_src_dport_ltm	-0.094091	-0.068373	-0.086695	-0.026490	-0.052135	0.35358
ct_dst_sport_ltm	-0.093923	-0.072484	-0.094267	-0.027281	-0.056901	0.39072
is_ftp_login	0.020641	0.009951	0.013491	-0.004515	-0.010460	-0.06814
ct_ftp_cmd	0.020641	0.009951	0.013491	-0.004515	-0.010460	-0.06814
ct_flw_http_mthd	0.024743	0.006084	0.047974	-0.002185	0.051403	-0.10929
is_sm_ips_ports	0.035370	-0.017770	-0.021765	-0.006367	-0.013147	-0.07294

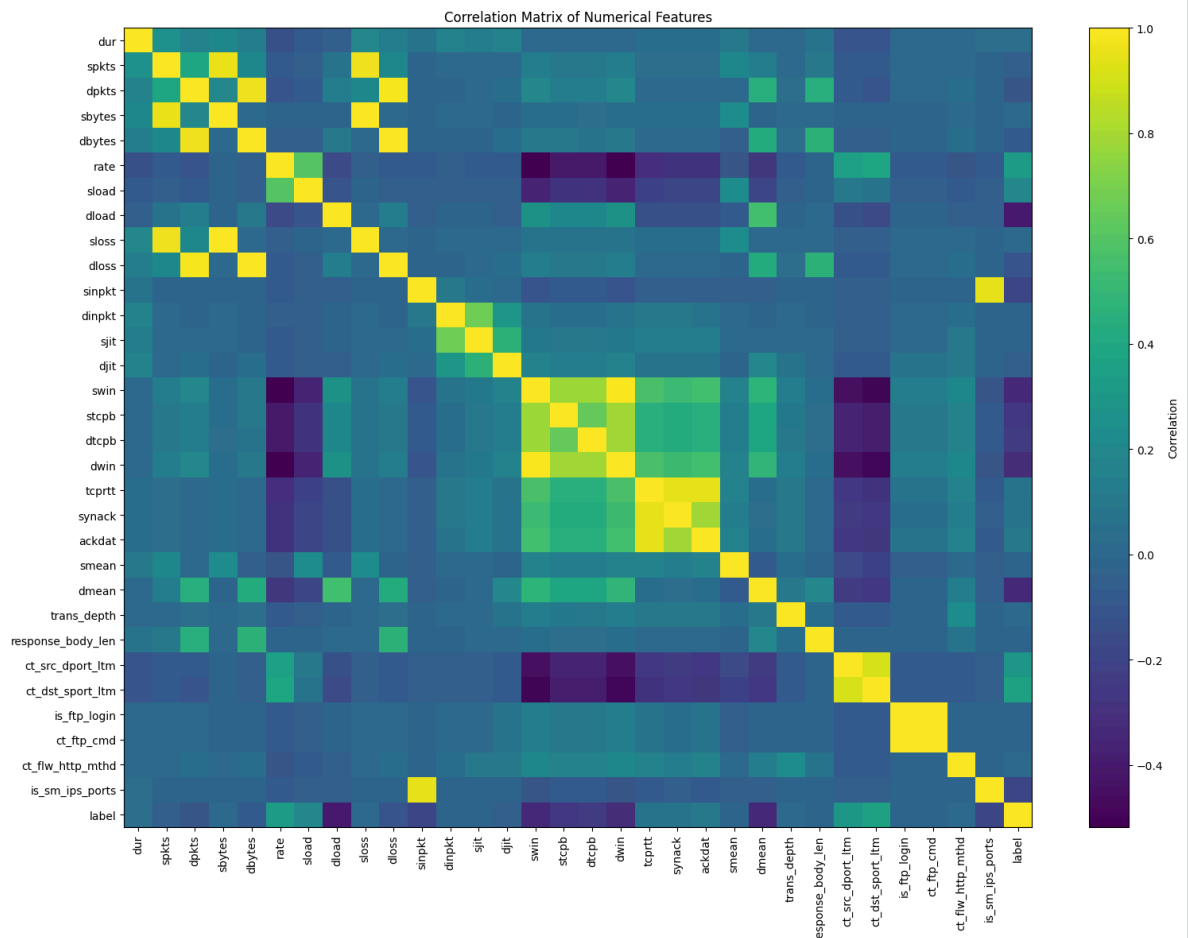
```
import matplotlib.pyplot as plt

plt.figure(figsize=(16, 12))
plt.imshow(correlation, aspect='auto')

plt.colorbar(label='Correlation')
plt.title('Correlation Matrix of Numerical Features')

plt.xticks(range(len(correlation.columns)), correlation.columns, rotation=90)
plt.yticks(range(len(correlation.columns)), correlation.columns)

plt.tight_layout()
plt.show()
```



```
# Correlation of numerical features with the target label
label_corr = correlation['label'].sort_values(ascending=False)

print("Features correlated with Attack Label:")
print(label_corr)
```

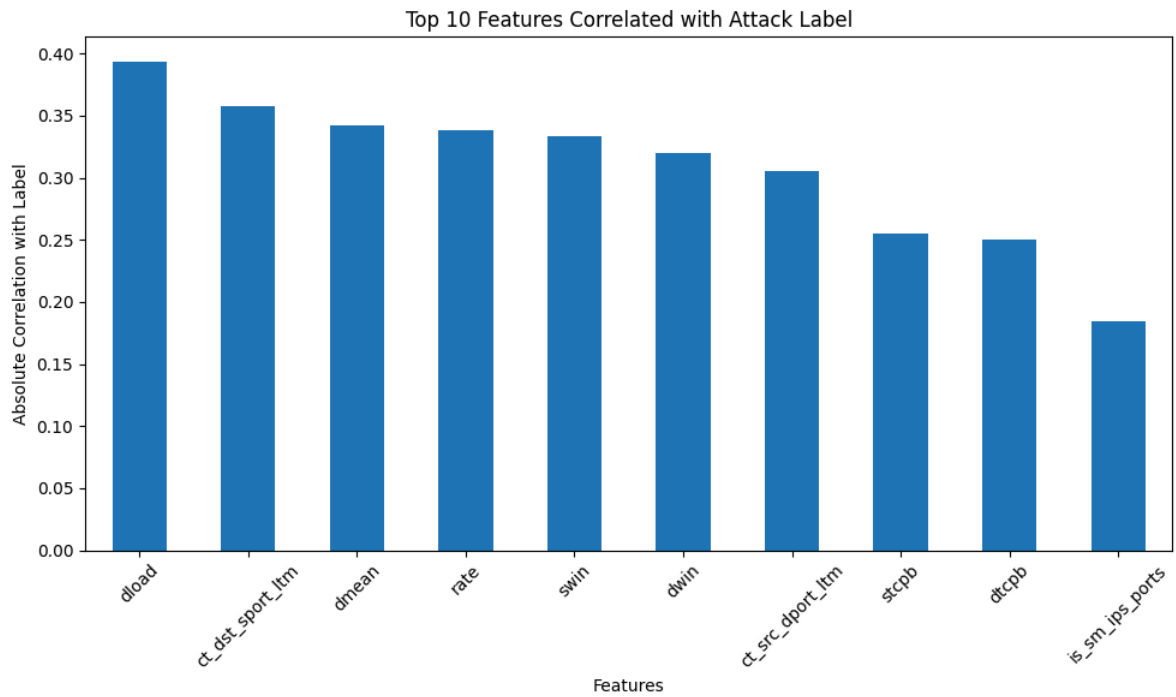
```
Features correlated with Attack Label:
label          1.000000
ct_dst_sport_ltm 0.357213
rate           0.337979
ct_src_dport_ltm 0.305579
sload          0.182870
ackdat         0.097364
```

```
tcprtt      0.081584
synack      0.058299
dur         0.036175
sbytes      0.018576
ct_flw_http_mthd  0.015800
trans_depth 0.010801
sloss      -0.000640
sjit        -0.007069
smean       -0.010798
ct_ftp_cmd  -0.011055
is_ftp_login -0.011055
response_body_len -0.021361
dinpkt      -0.022887
spkts       -0.052178
djit        -0.060870
dbytes      -0.076871
dloss       -0.094685
dpkts       -0.118591
sinpkt      -0.176110
is_sm_ips_ports -0.184679
dtcpb       -0.250340
stcpb       -0.255006
dwin        -0.319626
swin        -0.333633
dmean       -0.341806
dload       -0.393739
Name: label, dtype: float64
```

```
# Remove label itself and get top correlations by absolute value
top_corr = correlation['label'].drop('label').abs().sort_values(ascending=Fa

plt.figure(figsize=(10, 6))
top_corr.plot(kind='bar')

plt.title('Top 10 Features Correlated with Attack Label')
plt.xlabel('Features')
plt.ylabel('Absolute Correlation with Label')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```
# Check unique values in important categorical columns
categorical_cols = ['proto', 'service', 'state', 'attack_cat']
```

```
for col in categorical_cols:
    print(f"\n--- {col} ---")
    print("Unique values:", df[col].nunique())
    print(df[col].value_counts().head(10))
```

```
--- proto ---
Unique values: 133
proto
tcp      79946
udp      63283
unas     12084
arp       2859
ospf      2595
sctp      1150
any        300
gre        225
swipe      201
mobile     201
Name: count, dtype: int64
```

```
--- service ---
Unique values: 13
service
-          94168
dns        47294
http       18724
smtp       5058
ftp-data   3995
ftp        3428
ssh        1302
pop3       1105
```



```

dhcp          94
snmp          80
Name: count, dtype: int64

```

```

--- state ---

```

```

Unique values: 9

```

```

state

```

```

INT      82275

```

```

FIN      77825

```

```

CON      13152

```

```

REQ       1991

```

```

RST        83

```

```

ECO        12

```

```

PAR         1

```

```

URN         1

```

```

no          1

```

```

Name: count, dtype: int64

```

```

--- attack_cat ---

```

```

Unique values: 10

```

```

attack_cat

```

```

Normal          56000

```

```

Generic          40000

```

```

Exploits         33393

```

```

Fuzzers          18184

```

```

DoS              12264

```

```

Reconnaissance   10491

```

```

Analysis         2000

```

```

Backdoor         1746

```

```

Shellcode        1133

```

```

Worms            130

```

```

import os

```

```

print(os.listdir('/content'))

```

```

['.config', 'drive', 'UNSW_NB15_training-set.parquet', 'Botnet-Friday-no-meta

```

```

import pandas as pd

```

```

df_cic = pd.read_parquet('/content/UNSW_NB15_training-set.parquet')

```

```

display(df_cic.head())

```

	dur	proto	service	state	spkts	dpkts	sbytes	dbytes	rate	
0	0.121478	tcp	-	FIN	6	4	258	172	74.087486	14
1	0.649902	tcp	-	FIN	14	38	734	42014	78.473373	8
2	1.623129	tcp	-	FIN	8	16	364	13186	14.170161	1
3	1.681642	tcp	ftp	FIN	12	12	628	770	13.677108	2
4	0.449454	tcp	-	FIN	10	6	534	268	33.373825	8

5 rows × 36 columns

```
import os
print(os.listdir('/content'))
```

```
['.config', 'drive', 'UNSW_NB15_training-set.parquet', 'Botnet-Friday-no-meta
```

```
import pandas as pd

df_cic = pd.read_parquet(
    '/content/DDoS-Friday-no-metadata.parquet'
)

df_cic.head()
```

	Protocol	Flow Duration	Total Fwd Packets	Total Backward Packets	Fwd Packets Length Total	Bwd Packets Length Total	Fwd Packet Length Max	Fwd Packet Length Min	Pa Le
0	6	3	2	0	12	0	6	6	
1	6	109	1	1	6	6	6	6	
2	6	52	1	1	6	6	6	6	
3	6	34	1	1	6	6	6	6	
4	6	3	2	0	12	0	6	6	

5 rows × 78 columns

```
print("Dataset Shape:", df_cic.shape)

print("\nColumn Names:")
print(df_cic.columns.tolist())

df_cic.info()
```

Dataset Shape: (221264, 78)

```

Column Names:
['Protocol', 'Flow Duration', 'Total Fwd Packets', 'Total Backward Packets',
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 221264 entries, 0 to 221263
Data columns (total 78 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Protocol                              221264 non-null int8
1   Flow Duration                         221264 non-null int32
2   Total Fwd Packets                     221264 non-null int16
3   Total Backward Packets                221264 non-null int16
4   Fwd Packets Length Total              221264 non-null int32
5   Bwd Packets Length Total              221264 non-null int32
6   Fwd Packet Length Max                 221264 non-null int16
7   Fwd Packet Length Min                 221264 non-null int16
8   Fwd Packet Length Mean                221264 non-null float32
9   Fwd Packet Length Std                 221264 non-null float32
10  Bwd Packet Length Max                  221264 non-null int16
11  Bwd Packet Length Min                  221264 non-null int16
12  Bwd Packet Length Mean                 221264 non-null float32
13  Bwd Packet Length Std                  221264 non-null float32
14  Flow Bytes/s                           221264 non-null float64
15  Flow Packets/s                         221264 non-null float64
16  Flow IAT Mean                          221264 non-null float32
17  Flow IAT Std                           221264 non-null float32
18  Flow IAT Max                           221264 non-null int32
19  Flow IAT Min                           221264 non-null int32
20  Fwd IAT Total                          221264 non-null int32
21  Fwd IAT Mean                           221264 non-null float32
22  Fwd IAT Std                            221264 non-null float32
23  Fwd IAT Max                            221264 non-null int32
24  Fwd IAT Min                            221264 non-null int32
25  Bwd IAT Total                          221264 non-null int32
26  Bwd IAT Mean                           221264 non-null float32
27  Bwd IAT Std                            221264 non-null float32
28  Bwd IAT Max                            221264 non-null int32
29  Bwd IAT Min                            221264 non-null int32
30  Fwd PSH Flags                          221264 non-null int8
31  Bwd PSH Flags                          221264 non-null int8
32  Fwd URG Flags                          221264 non-null int8
33  Bwd URG Flags                          221264 non-null int8
34  Fwd Header Length                      221264 non-null int32
35  Bwd Header Length                      221264 non-null int32
36  Fwd Packets/s                          221264 non-null float32
37  Bwd Packets/s                          221264 non-null float32
38  Packet Length Min                      221264 non-null int16
39  Packet Length Max                      221264 non-null int16
40  Packet Length Mean                     221264 non-null float32
41  Packet Length Std                      221264 non-null float32
42  Packet Length Variance                 221264 non-null float32
43  FIN Flag Count                         221264 non-null int8
44  SYN Flag Count                         221264 non-null int8
45  RST Flag Count                         221264 non-null int8
46  PSH Flag Count                         221264 non-null int8
47  ACK Flag Count                         221264 non-null int8

```

```

print("Label Distribution:")
print(df_cic['Label'].value_counts())

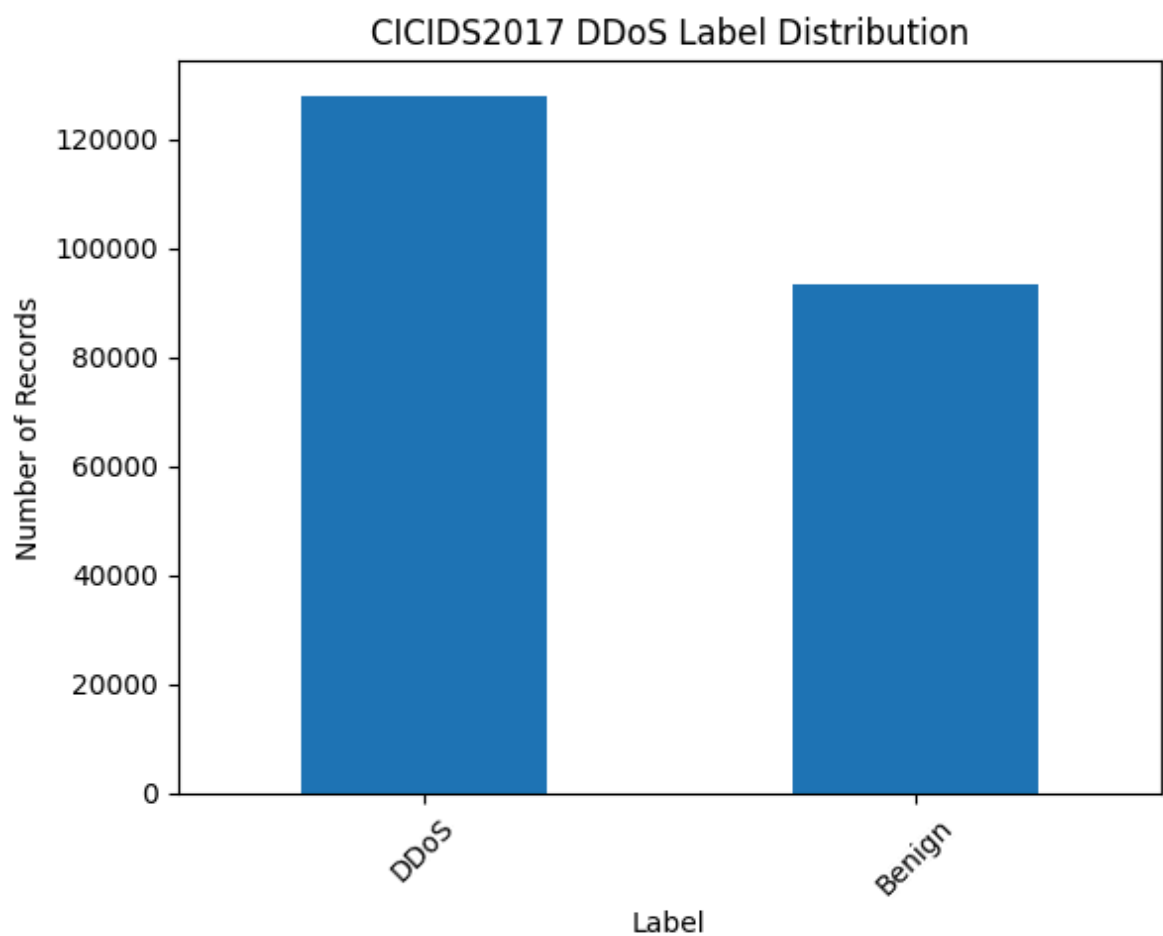
```

```
Label Distribution:  
Label  
DDoS      128014  
Benign     93250  
Name: count, dtype: int64
```

```
print("\nPercentage Distribution:")  
print(df_cic['Label'].value_counts(normalize=True) * 100)
```

```
Percentage Distribution:  
Label  
DDoS      57.855774  
Benign     42.144226  
Name: proportion, dtype: float64
```

```
import matplotlib.pyplot as plt  
  
df_cic['Label'].value_counts().plot(kind='bar')  
  
plt.title('CICIDS2017 DDoS Label Distribution')  
plt.xlabel('Label')  
plt.ylabel('Number of Records')  
plt.xticks(rotation=45)  
plt.show()
```



```
# Check missing values
print("Missing Values:")
print(df_cic.isnull().sum().sum())

# Check duplicate rows
print("\nDuplicate Rows:")
print(df_cic.duplicated().sum())
```

Missing Values:

0

Duplicate Rows:

0

```
display(df_cic.describe())
```

	Protocol	Flow Duration	Total Fwd Packets	Total Backward Packets	Fwd Packet Length Total
count	221264.000000	2.212640e+05	221264.000000	221264.000000	221264.000000
mean	7.541918	1.657047e+07	4.931069	4.647882	957.07463
std	3.822113	3.175633e+07	15.571754	21.967669	3279.60438
min	0.000000	-1.000000e+00	1.000000	0.000000	0.000000
25%	6.000000	9.032200e+04	2.000000	1.000000	26.000000
50%	6.000000	1.614922e+06	3.000000	4.000000	30.000000
75%	6.000000	9.018368e+06	5.000000	6.000000	62.000000
max	17.000000	1.199999e+08	1932.000000	2942.000000	183012.000000

8 rows × 77 columns

```
print("Dataset Shape:", df_cic.shape)
```

```
print("\nData Types:")
print(df_cic.dtypes)
```

Dataset Shape: (221264, 78)

Data Types:

```
Protocol                int8
Flow Duration           int32
Total Fwd Packets       int16
Total Backward Packets  int16
Fwd Packets Length Total int32
...
Idle Mean               float32
Idle Std                float32
Idle Max                int32
Idle Min                int32
```

Label
Length: 78, dtype: object

```
# Select only numerical columns
numeric_cic = df_cic.select_dtypes(include=['number'])

# Create correlation matrix
correlation_cic = numeric_cic.corr()

print("Correlation Matrix:")
display(correlation_cic)
```

Correlation Matrix:

	Protocol	Flow Duration	Total Fwd Packets	Total Backward Packets	Fwd Packets Length Total	Bwd Packets Length Total	Pack Len
Protocol	1.000000	-0.192281	-0.077245	-0.052540	-0.106627	-0.059995	-0.1082
Flow Duration	-0.192281	1.000000	0.206426	0.143927	0.296066	0.087440	0.2789
Total Fwd Packets	-0.077245	0.206426	1.000000	0.956805	0.173563	0.938530	0.0409
Total Backward Packets	-0.052540	0.143927	0.956805	1.000000	0.162751	0.970427	0.0460
Fwd Packets Length Total	-0.106627	0.296066	0.173563	0.162751	1.000000	0.066678	0.8522
...	...	...	...	...	...	...	...
Active Min	-0.091170	0.412797	0.086948	0.062822	0.170272	0.046176	0.1557
Idle Mean	-0.167102	0.871237	0.109360	0.064089	0.259337	0.040923	0.2703
Idle Std	-0.114777	0.629855	0.055845	0.027680	0.215578	0.019240	0.2293
Idle Max	-0.174257	0.918807	0.107983	0.061442	0.282977	0.039702	0.2963
Idle Min	-0.131481	0.670765	0.093507	0.056812	0.186884	0.035664	0.1933

77 rows × 77 columns

```
import matplotlib.pyplot as plt

plt.figure(figsize=(16, 12))

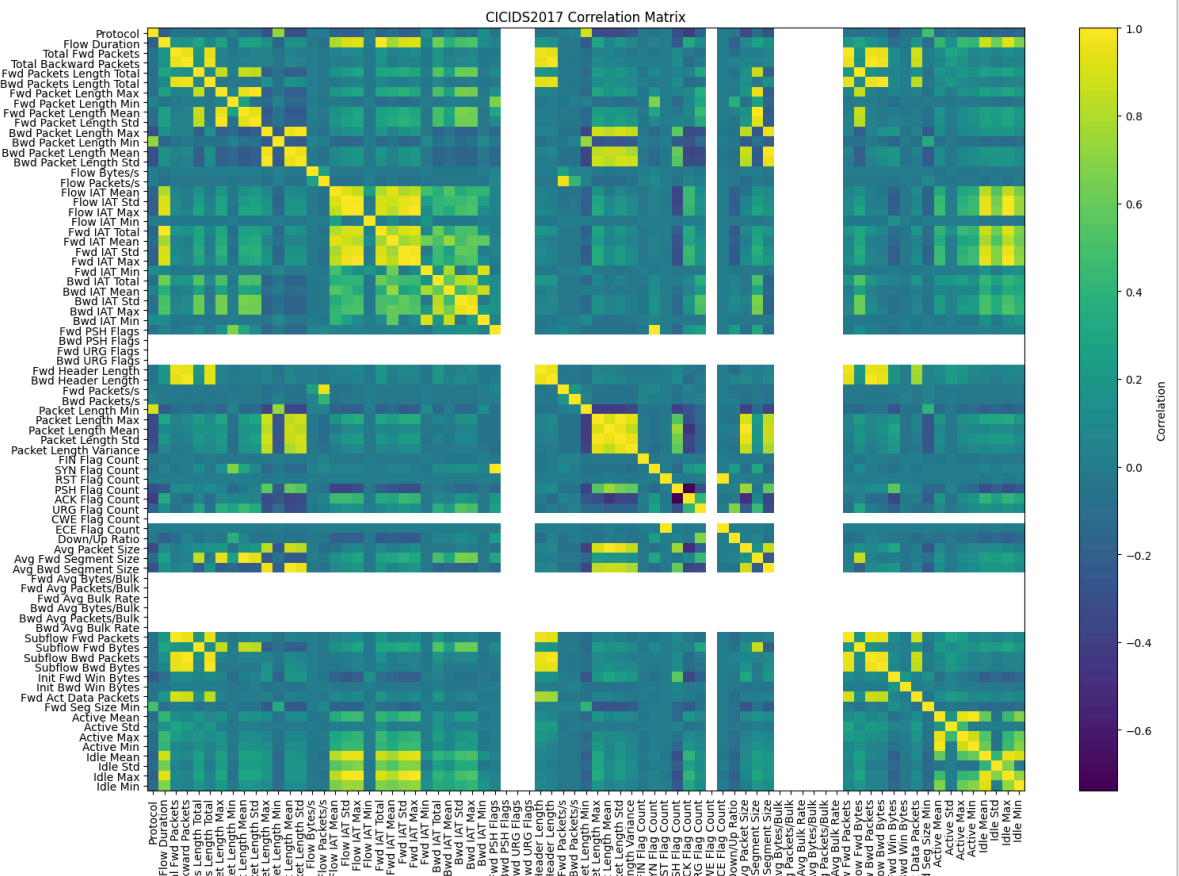
plt.imshow(correlation_cic, aspect='auto')
plt.colorbar(label='Correlation')
```

```
plt.title('CICIDS2017 Correlation Matrix')

plt.xticks(
    range(len(correlation_cic.columns)),
    correlation_cic.columns,
    rotation=90
)

plt.yticks(
    range(len(correlation_cic.columns)),
    correlation_cic.columns
)

plt.tight_layout()
plt.show()
```



```
import matplotlib.pyplot as plt

plt.figure(figsize=(16, 12))

plt.imshow(correlation_cic, aspect='auto')
plt.colorbar(label='Correlation')

plt.title('CICIDS2017 Correlation Matrix')

plt.xticks(
```

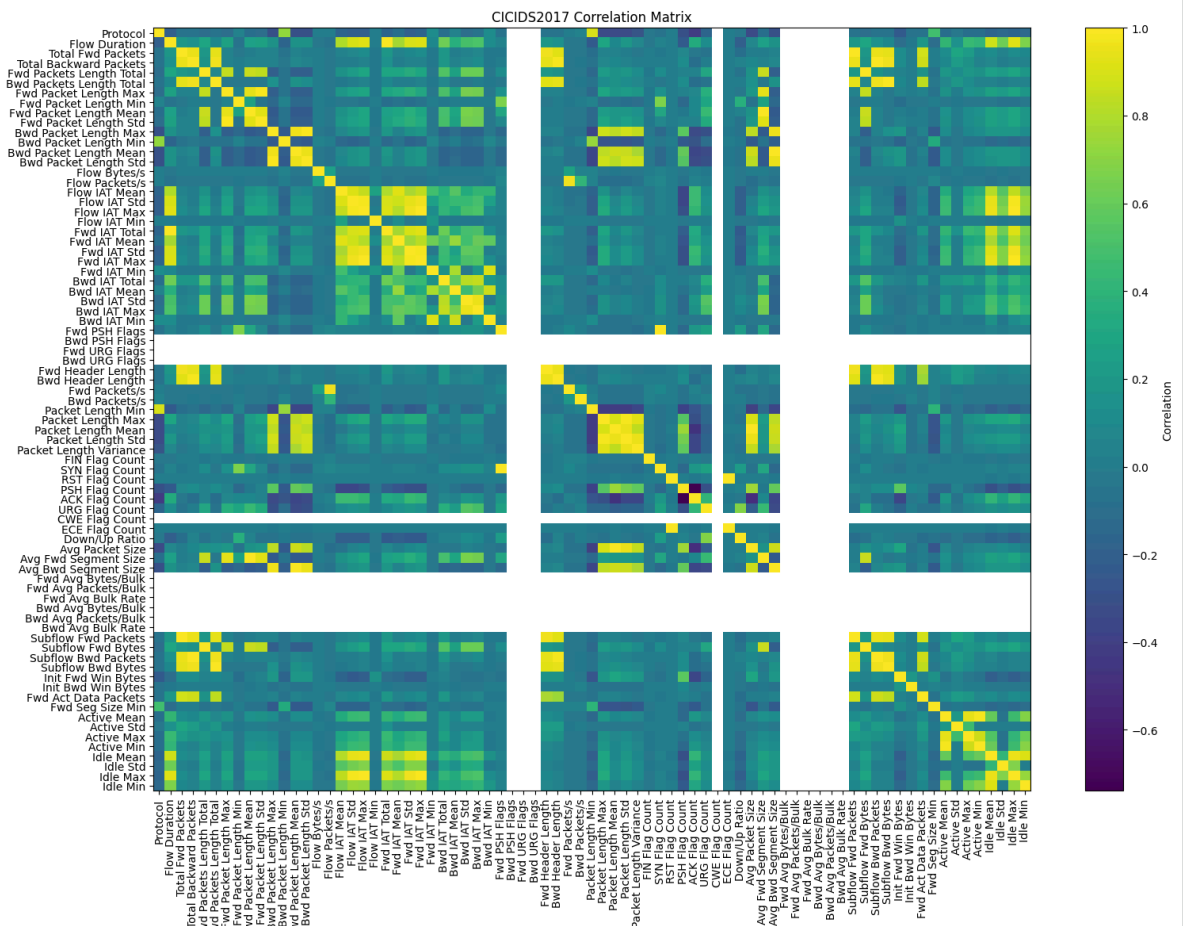
```

range(len(correlation_cic.columns)),
correlation_cic.columns,
rotation=90
)

plt.xticks(
range(len(correlation_cic.columns)),
correlation_cic.columns
)

plt.tight_layout()
plt.show()

```



```

# Display basic statistics for important numerical features
important_features = [
    'Flow Duration',
    'Total Fwd Packets',
    'Total Backward Packets',
    'Total Length of Fwd Packets',
    'Total Length of Bwd Packets',
    'Flow Bytes/s',
    'Flow Packets/s'
]

# Show only columns that actually exist
existing_features = [col for col in important_features if col in df_cic.colu

```



```
display(df_cic[existing_features].describe())
```

	Flow Duration	Total Fwd Packets	Total Backward Packets	Flow Bytes/s	Flow Packets/s
count	2.212640e+05	221264.000000	221264.000000	2.212640e+05	2.212640e+05
mean	1.657047e+07	4.931069	4.647882	2.732280e+05	8.107003e+03
std	3.175633e+07	15.571754	21.967669	8.222828e+06	8.280048e+04
min	-1.000000e+00	1.000000	0.000000	-1.200000e+07	-2.000000e+06
25%	9.032200e+04	2.000000	1.000000	1.171832e+01	6.060721e-01
50%	1.614922e+06	3.000000	4.000000	8.158733e+02	4.883782e+00
75%	9.018368e+06	5.000000	6.000000	1.870054e+04	6.092128e+01
max	1.199999e+08	1932.000000	2942.000000	2.070000e+09	3.000000e+06

```
import pandas as pd

files = [
    '/content/Botnet-Friday-no-metadata.parquet',
    '/content/Benign-Monday-no-metadata.parquet',
    '/content/DDoS-Friday-no-metadata.parquet',
    '/content/DoS-Wednesday-no-metadata.parquet',
    '/content/Portscan-Friday-no-metadata.parquet',
    '/content/Infiltration-Thursday-no-metadata.parquet'
]

# Load and combine all datasets
dfs = [pd.read_parquet(file) for file in files]

df_cic_all = pd.concat(dfs, ignore_index=True)

print("Combined Dataset Shape:", df_cic_all.shape)

print("\nAttack Distribution:")
print(df_cic_all['Label'].value_counts())
```

Combined Dataset Shape: (1768276, 78)

Attack Distribution:

Label	
Benign	1443077
DoS Hulk	172846
DDoS	128014
DoS GoldenEye	10286
DoS slowloris	5385
DoS Slowhttptest	5228
PortScan	1956
Bot	1437

```
Infiltration          36
Heartbleed            11
Name: count, dtype: int64
```

```
import matplotlib.pyplot as plt

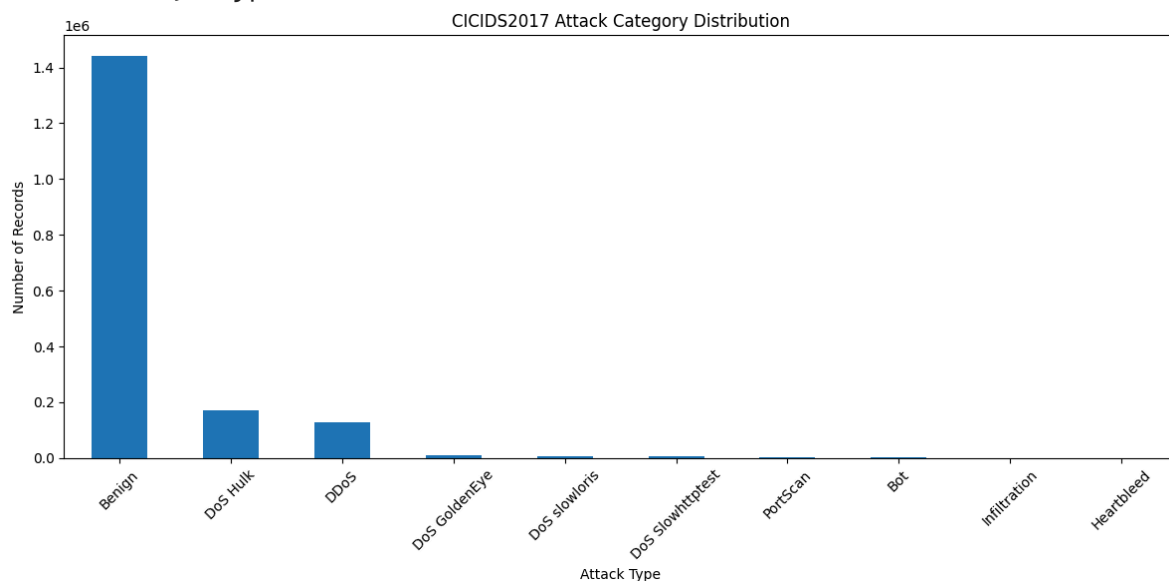
# Count each attack type
attack_counts = df_cic_all['Label'].value_counts()

print(attack_counts)

# Create graph
plt.figure(figsize=(12, 6))
attack_counts.plot(kind='bar')

plt.title('CICIDS2017 Attack Category Distribution')
plt.xlabel('Attack Type')
plt.ylabel('Number of Records')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
Label
Benign          1443077
DoS Hulk        172846
DDoS            128014
DoS GoldenEye   10286
DoS slowloris   5385
DoS Slowhttptest 5228
PortScan        1956
Bot             1437
Infiltration     36
Heartbleed       11
Name: count, dtype: int64
```



```
print("Total Rows:", df_cic_all.shape[0])
print("Total Columns:", df_cic_all.shape[1])
```

```
print("\nMissing Values:")
print(df_cic_all.isnull().sum().sum())

print("\nDuplicate Rows:")
print(df_cic_all.duplicated().sum())
```

Total Rows: 1768276
Total Columns: 78

Missing Values:
0

Duplicate Rows:
46441

```
import pandas as pd

comparison = pd.DataFrame({
    'Feature': [
        'Dataset',
        'Records',
        'Number of Columns',
        'Attack Types',
        'Missing Values',
        'Duplicates',
        'Target Column'
    ],
    'UNSW-NB15': [
        'UNSW-NB15',
        175341,
        36,
        'Generic, Exploits, Fuzzers, DoS, Reconnaissance, Analysis, Backdoor',
        '0',
        78519,
        'label / attack_cat'
    ],
    'CICIDS2017': [
        'CICIDS2017',
        df_cic_all.shape[0],
        df_cic_all.shape[1],
        'Benign, DDoS, DoS, Botnet, PortScan, Infiltration',
        df_cic_all.isnull().sum().sum(),
        df_cic_all.duplicated().sum(),
        'Label'
    ]
})

display(comparison)
```

	Feature	UNSW-NB15	CICIDS2017
0	Dataset	UNSW-NB15	CICIDS2017
1	Records	175341	1768276
2	Number of Columns	36	78
3	Attack Types	Generic, Exploits, Fuzzers, DoS, Reconnaissance...	Benign, DDoS, DoS, Botnet, PortScan, Infiltration
4	Missing Values	0	0
5	Duplicates	78519	46441
6	Target Column	label / attack_cat	Label

```
import pandas as pd

# Load UNSW-NB15 dataset
df = pd.read_parquet('/content/UNSW_NB15_training-set.parquet')

print("Dataset loaded successfully!")
print("Shape:", df.shape)

display(df.head())
```

```
-----
FileNotFoundError                                Traceback (most recent call last)
/tmp/ipykernel_3816/1178171500.py in <cell line: 0>()
      2
      3 # Load UNSW-NB15 dataset
----> 4 df = pd.read_parquet('/content/UNSW_NB15_training-set.parquet')
      5
      6 print("Dataset loaded successfully!")

----- 3 frames -----
/usr/local/lib/python3.13/dist-packages/pandas/io/common.py in
get_handle(path_or_buf, mode, encoding, compression, memory_map, is_text,
errors, storage_options)
    880     else:
    881         # Binary mode
--> 882         handle = open(handle, ioargs.mode)
    883         handles.append(handle)
    884

FileNotFoundError: [Errno 2] No such file or directory:
'/content/UNSW_NB15_training-set.parquet'
```

Next steps: [Explain error](#)

```
import os
```

```
print(os.listdir('/content'))
```

```
['.config', 'drive', 'sample_data']
```

```
import pandas as pd
import os

# List all files in the /content directory
files_in_content = os.listdir('/content')

# Filter for CICIDS2017 related parquet files (excluding UNSW-NB15 files)
# Based on the file list provided in the kernel state
cicids_parquet_files = [
    f'/content/{f}' for f in files_in_content
    if f.endswith('.parquet') and 'UNSW-NB15' not in f
]

if not cicids_parquet_files:
    print("No CICIDS2017 parquet files found in /content.")
else:
    print("Found CICIDS2017 files:", cicids_parquet_files)

    # Load and combine all datasets
    dfs = [pd.read_parquet(file) for file in cicids_parquet_files]

    df_cic_all = pd.concat(dfs, ignore_index=True)

    print("\nCombined CICIDS2017 Dataset Shape:", df_cic_all.shape)
    print("Combined CICIDS2017 Dataset loaded successfully!")
    display(df_cic_all.head())
```

Found CICIDS2017 files: ['/content/Botnet-Friday-no-metadata.parquet', '/cont

Combined CICIDS2017 Dataset Shape: (2313810, 78)

Combined CICIDS2017 Dataset loaded successfully!

	Protocol	Flow Duration	Total Fwd Packets	Total Backward Packets	Fwd Packets Length Total	Bwd Packets Length Total	Fwd Packet Length Max	Fwd Packet Length Min	F
0	6	112740690	32	16	6448	1152	403	0	
1	6	112740560	32	16	6448	5056	403	0	
2	0	113757377	545	0	0	0	0	0	
3	17	100126	22	0	616	0	28	28	
4	0	54760	4	0	0	0	0	0	

5 rows × 78 columns

```
# =====
# UNSW-NB15 - Accuracy, Precision, Recall, F1
# =====
```

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)

# -----
# 1. Find and load the uploaded dataset
# -----
import os

files_in_colab = os.listdir('/content')
print("Files available:", files_in_colab)

parquet_files = [f for f in files_in_colab if f.endswith('.parquet')]

if len(parquet_files) == 0:
    raise FileNotFoundError("No .parquet dataset found in /content")

file_path = '/content/' + parquet_files[0]

print("\nLoading:", file_path)

df = pd.read_parquet(file_path)

print("Dataset shape:", df.shape)
print("Dataset loaded successfully!")

# -----
# 2. Remove duplicate rows
# -----
df = df.drop_duplicates().copy()

print("\nAfter removing duplicates:", df.shape)

# -----
# 3. Separate features and target
# -----
y = df['label']

X = df.drop(columns=['label', 'attack_cat'], errors='ignore')
```

```
# -----
# 4. Identify categorical columns
# -----
categorical_columns = X.select_dtypes(
    include=['object', 'category']
).columns.tolist()

numerical_columns = X.select_dtypes(
    exclude=['object', 'category']
).columns.tolist()

print("\nCategorical columns:", categorical_columns)
print("Numerical columns:", len(numerical_columns))

# -----
# 5. One-hot encode categorical columns
# -----
preprocessor = ColumnTransformer(
    transformers=[
        ('cat',
         OneHotEncoder(handle_unknown='ignore',
                        sparse_output=False),
         categorical_columns)
    ],
    remainder='passthrough'
)

X = preprocessor.fit_transform(X)

X = X.astype(np.float32)

print("\nFinal feature shape:", X.shape)

# -----
# 6. Train-test split
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])

# -----
# 7. Train Random Forest
# -----
model = RandomForestClassifier(
```

```

        n_estimators=100,
        random_state=42,
        n_jobs=-1
    )

print("\nTraining model...")

model.fit(X_train, y_train)

print("Model training completed!")

# -----
# 8. Predictions
# -----
y_pred = model.predict(X_test)

# -----
# 9. Calculate metrics
# -----
accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(
    y_test,
    y_pred,
    average='binary',
    zero_division=0
)

recall = recall_score(
    y_test,
    y_pred,
    average='binary',
    zero_division=0
)

f1 = f1_score(
    y_test,
    y_pred,
    average='binary',
    zero_division=0
)

# -----
# 10. Display final results
# -----
print("\n=====")
print("          MODEL PERFORMANCE")
print("=====")

print(f"Accuracy : {accuracy:.4f} ({accuracy*100:.2f}%)")
print(f"Precision : {precision:.4f} ({precision*100:.2f}%)")
print(f"Recall : {recall:.4f} ({recall*100:.2f}%)")

```



```

print(f"F1-Score : {f1:.4f} ({f1*100:.2f}%)")

print("\n=====")
print("Classification Report")
print("=====")

print(classification_report(y_test, y_pred, zero_division=0))

print("\n=====")
print("Confusion Matrix")
print("=====")

print(confusion_matrix(y_test, y_pred))

```

Files available: ['.config', 'UNSW_NB15_training-set.parquet', 'UNSW_NB15_tes

Loading: /content/UNSW_NB15_training-set.parquet

Dataset shape: (175341, 36)

Dataset loaded successfully!

After removing duplicates: (96822, 36)

Categorical columns: ['proto', 'service', 'state']

Numerical columns: 31

Final feature shape: (96822, 186)

Training samples: 77457

Testing samples: 19365

Training model...

Model training completed!

=====

MODEL PERFORMANCE

=====

Accuracy : 0.9196 (91.96%)

Precision : 0.8992 (89.92%)

Recall : 0.9434 (94.34%)

F1-Score : 0.9207 (92.07%)

=====

Classification Report

=====

	precision	recall	f1-score	support
0	0.94	0.90	0.92	9779
1	0.90	0.94	0.92	9586
accuracy			0.92	19365
macro avg	0.92	0.92	0.92	19365
weighted avg	0.92	0.92	0.92	19365

=====

Confusion Matrix

```
=====
[[8765 1014]
 [ 543 9043]]
```

```
print("Missing Values:")
print(df.isnull().sum().sum())
```

```
print("\nDuplicate Rows:")
print(df.duplicated().sum())
```

```
Missing Values:
0
```

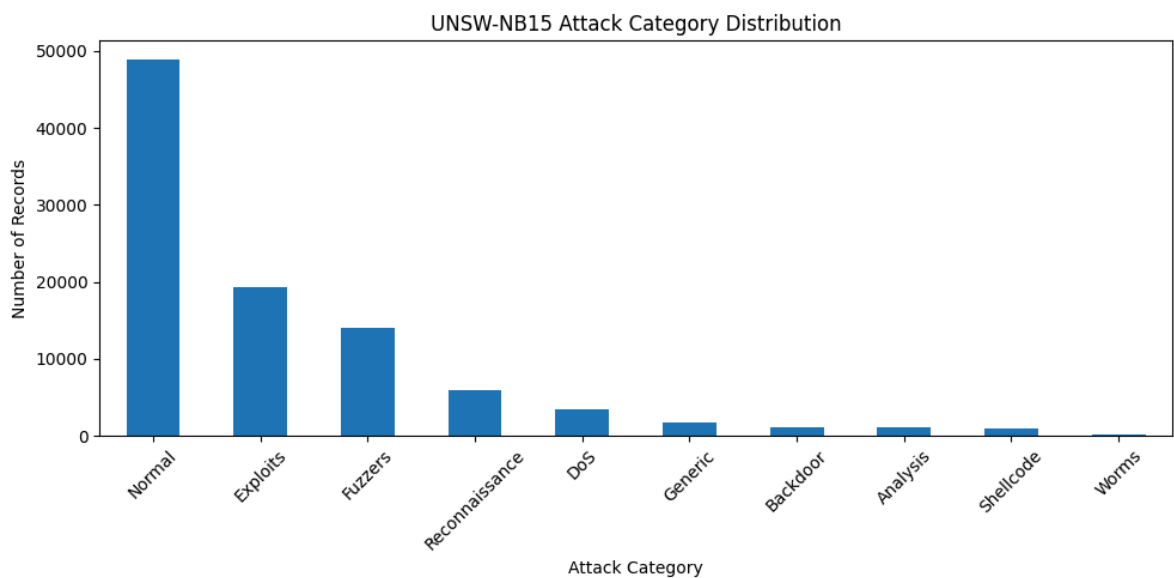
```
Duplicate Rows:
0
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))

df['attack_cat'].value_counts().plot(kind='bar')

plt.title('UNSW-NB15 Attack Category Distribution')
plt.xlabel('Attack Category')
plt.ylabel('Number of Records')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

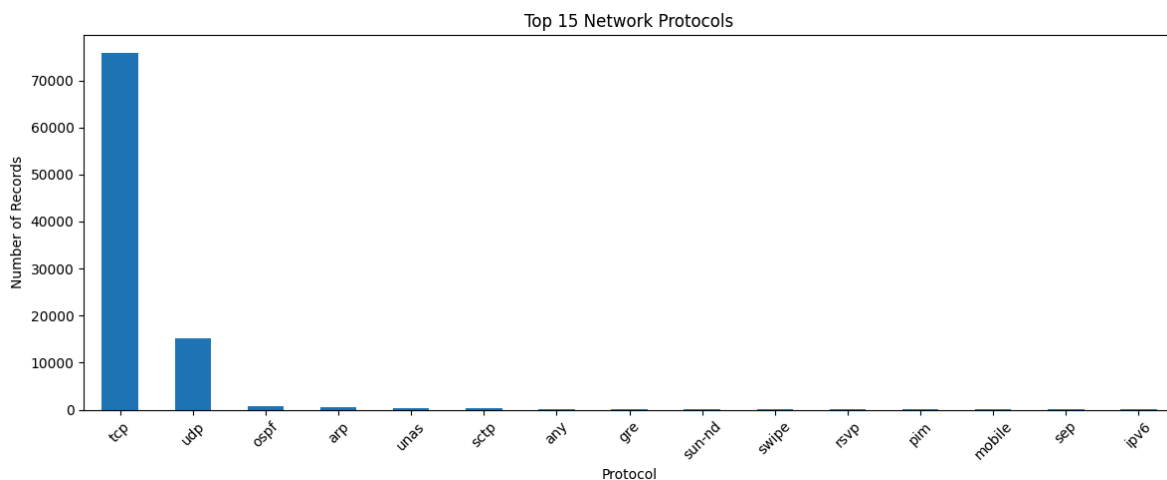


```
plt.figure(figsize=(12,5))

df['proto'].value_counts().head(15).plot(kind='bar')

plt.title('Top 15 Network Protocols')
plt.xlabel('Protocol')
plt.ylabel('Number of Records')
plt.xticks(rotation=45)
```

```
plt.tight_layout()  
plt.show()
```



```
label_percentage = df['label'].value_counts(normalize=True) * 100
```

```
print(label_percentage)
```

```
plt.figure(figsize=(6,5))
```

```
label_percentage.plot(kind='pie', autopct='%1.1f%%')
```

```
plt.title('Normal vs Attack Traffic')
```

```
plt.ylabel('')
```

```
plt.show()
```

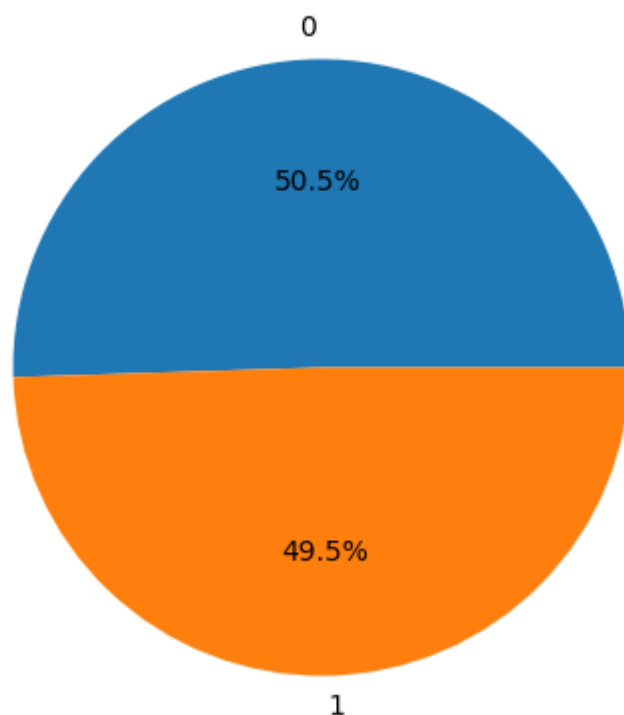
```
label
```

```
0    50.498854
```

```
1    49.501146
```

```
Name: proportion, dtype: float64
```

Normal vs Attack Traffic



```
print("===== DATASET COMPARISON =====")

print("\nUNSW-NB15")
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])

print("\nCICIDS2017")
print("Rows:", df_cic_all.shape[0])
print("Columns:", df_cic_all.shape[1])
```

```
===== DATASET COMPARISON =====
```

```
UNSW-NB15
Rows: 96822
Columns: 36
```

```
CICIDS2017
Rows: 2313810
Columns: 78
```

```
from google.colab import sheets
sheet = sheets.InteractiveSheet(df=comparison)
```

```
-----
MessageError                                Traceback (most recent call last)
/tmp/ipykernel_3816/2967319991.py in <cell line: 0>()
      1 from google.colab import sheets
----> 2 sheet = sheets.InteractiveSheet(df=comparison)
```

```
----- 6 frames -----
/usr/local/lib/python3.13/dist-packages/google/colab/message.py in
read_reply_from_input(message_id, timeout_sec)
    101     ):
    102         if 'error' in reply:
--> 103             raise MessageError(reply['error'])
    104             return reply.get('data', None)
    105
```

```
MessageError: Error: credential propagation was unsuccessful
```

Next steps: [Explain error](#)

```
comparison = pd.DataFrame({
    'Dataset': ['UNSW-NB15', 'CICIDS2017'],
    'Rows': [df.shape[0], df_cic_all.shape[0]],
    'Columns': [df.shape[1], df_cic_all.shape[1]]
})

display(comparison)
```

	Dataset	Rows	Columns	
0	UNSW-NB15	96822	36	
1	CICIDS2017	2313810	78	

Next steps:

[Generate code with comparison](#)[New interactive sheet](#)

```
from google.colab import sheets
sheet = sheets.InteractiveSheet(df=comparison)
```

<https://docs.google.com/spreadsheets/d/1064-F1bMPE49jdCkZCDsSGz1Q0nIerZtXAelA>

File Edit View Insert Format Data Tools Extensions Help						
Q Menus     100%  \$ % .0  .00  123 : ^						
A1	Dataset					
	A	B	C	D	E	
79						
80						
81						
82						
83						
84						
85						
86						
87						
88						
89						
90						
91						
92						
93						
94						
95						
96						
97						
98						

  Sheet1 

```
# Check columns and data types
print("UNSW-NB15 Data Types:")
display(df.dtypes)

print("\nDataset Shape:")
print(df.shape)
```

UNSW-NB15 Data Types:

0

dur	float32
proto	category
service	category
state	category
spkts	int16
dpkts	int16
sbytes	int32
dbytes	int32
rate	float32
sload	float32